



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS**

A Project Report On
Project/Thesis Management System (TPMS)

Submitted By:

Prabesh BC (PUL080BCT054)
Shreeyut Thapa (PUL080BCT080)
Subesh Yadav (PUL080BCT084)

Submitted To:

Department of Electronics & Computer Engineering
In Partial Fulfillment of the Requirements for the Degree of
Bachelor of Engineering in Computer Engineering

**DEPARTMENT OF ELECTRONICS & COMPUTER
ENGINEERING**

Lalitpur, Nepal

August, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled “**Project/Thesis Management System (TPMS)**” submitted by **Prabesh BC, Shreeyut Thapa, and Subesh Yadau** in partial fulfillment of the requirements for the Bachelor’s degree in Electronics & Computer Engineering.

.....

Supervisor

Supervisor Full Name

Designation, Department of Electronics and
Computer Engineering,
Pulchowk Campus, IOE, TU.

.....

External Examiner

Examiner Full Name

Assistant Professor

Department of Electronics and Computer
Engineering,
Pulchowk Campus, IOE, TU.

Date of approval:

Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, Pulchowk Campus, and Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering for any use of the material of this project report. Copying, publication, or other use of this report for financial gain without the approval of the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head

Department of Electronics and Computer Engineering

Pulchowk Campus, Institute of Engineering, TU

Lalitpur, Nepal.

Acknowledgements

We would like to express our profound gratitude to our project supervisor, [**Supervisor Full Name**], Department of Electronics and Computer Engineering, Pulchowk Campus, for their continuous guidance, constructive feedback, and unwavering support throughout the entire development lifecycle of this project. Their technical insights and pedagogical direction were instrumental in shaping the architecture and quality of the Thesis and Project Management System.

We are deeply grateful to the **Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, Tribhuvan University**, for providing the academic framework, laboratory infrastructure, and institutional context that made this project possible.

We extend our sincere thanks to all faculty members and staff of DOECE who participated in User Acceptance Testing sessions and provided candid operational feedback that significantly improved the system's usability and functional coverage.

We are also thankful to our fellow classmates and student peers who volunteered their time to validate the student-facing group formation and document submission workflows.

Finally, we wish to express our deepest appreciation to our families for their constant moral support and encouragement throughout the duration of this program.

Abstract

The Project/Thesis Management System (TPMS) is a production-grade, role-aware web platform developed for the Department of Electronics and Computer Engineering (DOECE), Pulchowk Campus, Institute of Engineering, Tribhuvan University. The system digitalizes and automates the complete administrative and evaluative lifecycle of Bachelor Minor and Major projects and Master research theses across six academic programs (BCT, BEL, MSCSK, MSICE, MSDSA, MSNCS), replacing existing manual, paper-based workflows.

The platform enforces a five-role hierarchy (Maintainer, Coordinator, Supervisor, Student, External Examiner) governed by Role-Based Access Control (RBAC) and strict program-scoped degree isolation. A Global Engagement Guard prevents students from enrolling in multiple active project groups simultaneously, while an automated Conflict-of-Interest Filter ensures that assigned supervisors cannot serve as defense evaluators for their own projects. The system implements the DOECE evaluation schemes (Minor: 50 marks; Major: 100 marks; Master Thesis: 300 marks) through a dynamic rubric engine.

A server-side Puppeteer headless Chrome pipeline generates official A4 PDF grade sheets with automated number-to-words mark conversion. A SheetJS-powered Excel bulk import pipeline with pre-commit anomaly detection enables large-scale student onboarding. A program-scoped audit trail provides tamper-evident event logging for institutional accountability.

The system is implemented with React 18 and Vite 5 on the frontend, Node.js and Express.js on the backend, and PostgreSQL 16 managed via Prisma 5 ORM. Evaluation through unit testing (36 tests, 100% pass rate), integration testing, security testing, and User Acceptance Testing (average satisfaction 4.47/5.0) confirmed functional completeness and operational readiness. All API endpoints responded within 300 ms under test load, and PDF generation completed within 2 seconds.

Keywords: Academic Project Management, Role-Based Access Control, Evaluation Rubric Engine, PDF Grade Sheet Generation, Conflict-of-Interest Prevention, Node.js, React, PostgreSQL, Prisma ORM

Contents

Page of Approval	ii
Copyright	iii
Acknowledgements	iv
Abstract	v
Contents	ix
List of Figures	x
List of Tables	xi
List of Abbreviations	xii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	3
1.3.1 Main Objective	3
1.3.2 Specific Objectives	3
1.4 Scope and Limitations	4
1.4.1 Scope of the Project	4
1.4.2 Significance of the Project	4
1.5 Report Organization	5
2 Literature Review	7
2.1 Theoretical Background	7
2.1.1 Role-Based Access Control (RBAC) and Capability Models	7
2.1.2 RESTful Architecture and Stateless Web Services	7
2.1.3 Object-Relational Mapping (ORM) and Type-Safe Data Persistence	8
2.1.4 JSON Web Token (JWT) Authentication	8

2.1.5	Agile Scrum Framework in Applied Engineering	9
2.1.6	Automated Server-Side Document Rendering	9
2.2	Review of Related Works	9
2.2.1	General Learning Management Systems (LMS)	9
2.2.2	Electronic Thesis and Dissertation (ETD) Platforms	10
2.2.3	Academic Integrity and Submission Gateways	10
2.2.4	Academic Project Management Systems in Literature	10
2.3	Comparative Study	11
2.4	Gap Analysis and Research Motivation	11
3	Methodology and Requirement Analysis	13
3.1	Software Development Methodology	13
3.1.1	Scrum Workflow and Sprint Cycles	13
3.2	Requirement Analysis	14
3.2.1	Stakeholder Identification and Role Matrix	14
3.2.2	Functional Requirements	15
3.2.3	Non-Functional Requirements	16
3.3	Feasibility Study	17
3.3.1	Technical Feasibility	17
3.3.2	Operational Feasibility	17
3.3.3	Economic Feasibility	17
3.3.4	Schedule Feasibility	17
3.4	Technology Stack and Toolchain Justification	18
4	System Design and Architecture	19
4.1	High-Level System Architecture	19
4.2	Data Flow Diagrams (DFDs)	20
4.2.1	Context Diagram (Level 0)	20
4.2.2	Level 1 Data Flow Diagram	20
4.3	Use Case Diagram	21
4.4	Activity Diagram: Group Formation and Proposal Lifecycle	22
4.5	Database Entity-Relationship Diagram	23
4.6	Database Design	25
4.6.1	Data Dictionary	25
4.7	Security Architecture and Algorithmic Controls	26

4.7.1	Conflict-of-Interest Prevention	26
4.7.2	TPMS Evaluation Scheme	27
4.7.3	Global Multi-Project Engagement Guard	27
4.7.4	Institutional Email Domain Policy	27
5	Implementation	28
5.1	Backend Implementation	28
5.1.1	Server Entry Point and Middleware Pipeline	28
5.1.2	Authentication Module	28
5.1.3	Group Formation and Invitation Engine	29
5.1.4	Coordinator Response Matrix and Finalization	29
5.1.5	Evaluation Rubric Engine	29
5.1.6	PDF Grade Sheet Generation	30
5.1.7	Excel Bulk Import with Anomaly Detection	31
5.1.8	Program-Scoped Audit Logging	31
5.2	Frontend Implementation	31
5.2.1	Application Architecture and Routing	31
5.2.2	Authentication and Token Management	32
5.2.3	Coordinator Response Matrix View	32
5.2.4	Evaluation Rubric Panel	32
5.2.5	PDF Grade Sheet Preview and Export	33
5.2.6	Notification System	33
5.3	Sample API Route Reference	34
6	Results and Discussion	35
6.1	System Testing	35
6.1.1	Unit Testing	35
6.1.2	Integration Testing	36
6.1.3	Security Testing	36
6.1.4	User Acceptance Testing (UAT)	37
6.2	System Performance Evaluation	38
6.3	Functional Validation of Evaluation Metrics	40
6.4	Discussion	40

7	Limitations and Future Enhancements	41
7.1	Current Limitations	41
7.2	Future Enhancements	42
8	Conclusion	44
	References	45
	Appendices	48

List of Figures

4.1	Three-Tier System Architecture of TPMS	19
4.2	Level 0 Context Data Flow Diagram	20
4.3	Level 1 Data Flow Diagram	21
4.4	System Use Case Diagram	22
4.5	Activity Diagram: Group Formation and Proposal Submission Lifecycle .	23
4.6	Simplified Entity-Relationship Diagram of TPMS Database	24
4.7	Conflict-of-Interest Filter Algorithm Flow	26
4.8	TPMS Evaluation Scheme: Mark Distribution by Project Tier	27

List of Tables

2.1	Comparative Analysis of Academic Management Platforms	11
3.1	Agile Development Sprint Schedule and Deliverables	14
3.2	Stakeholder Personas and Role Responsibility Matrix	14
3.3	System Technology Stack and Architectural Justification	18
4.1	Data Dictionary: User Table	25
4.2	Data Dictionary: ProjectGroup Table	25
4.3	Data Dictionary: Evaluation Table	26
5.1	Key REST API Endpoints and Their Functions	34
6.1	Unit Test Summary	36
6.2	User Acceptance Testing Results by Role	37
6.3	API Response Time Benchmarks	39

List of Abbreviations

API	Application Programming Interface
BCT	Bachelor of Computer Engineering
BEI	Bachelor of Electronics, Communication and Information Engineering
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
COI	Conflict of Interest
DOECE	Department of Electronics and Computer Engineering
DFD	Data Flow Diagram
HTTP	Hypertext Transfer Protocol
IOE	Institute of Engineering
JWT	JSON Web Token
MSDSA	Master of Science in Data Science and Analytics
MSICE	Master of Science in Information and Communication Engineering
MSCSK	Master of Science in Computer Systems and Knowledge Engineering
MSNCS	Master of Science in Network and Cyber Security
MVC	Model-View-Controller
ORM	Object-Relational Mapping
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
TPMS	Thesis and Project Management System
TU	Tribhuvan University
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
VPS	Virtual Private Server

1. Introduction

1.1 Background

Academic project and thesis coursework represent pivotal milestones in engineering curricula at Tribhuvan University (TU), Institute of Engineering (IOE), Pulchowk Campus. Under the Department of Electronics and Computer Engineering (DOECE), undergraduate students enrolled in Bachelor of Computer Engineering (BCT) and Bachelor of Electronics, Communication and Information Engineering (BEI) undertake compulsory Minor and Major projects during their third and fourth academic years. Concurrently, postgraduate scholars across specialized Master of Science programs—including Computer Systems and Knowledge Engineering (MSCSK), Information and Communication Engineering (MSICE), Data Science and Analytics (MSDSA), and Network and Cyber Security (MSNCS)—execute multi-semester research theses.

Historically, the administrative, supervisory, and evaluative management of these academic initiatives has relied almost exclusively on manual, fragmented, and paper-intensive procedures. Departmental coordinators have had to physically gather proposal drafts, manually record group rosters into ad-hoc spreadsheets, collect printed mid-term and final reports, and distribute handwritten evaluation marking rubrics during formal defense sessions. As student enrollment and specialized postgraduate programs have grown, this legacy operational model has created significant administrative strain, operational bottlenecks, scheduling conflicts, and data inconsistencies.

Modern higher education institutions increasingly leverage specialized academic information systems to govern collaborative workflows, ensure institutional accountability, and maintain transparent audit trails [1, 2]. The **Thesis and Project Management System (TPMS)** is developed as an enterprise-grade, web-based platform engineered specifically to digitalize, govern, and streamline the complete lifecycle of Bachelor projects and Master theses at DOECE, Pulchowk Campus. By incorporating strict degree isolation, automated conflict-of-interest prevention, dynamic evaluation rubric computation, server-side grade sheet rendering, and program-scoped audit logging, TPMS replaces disparate manual processes with a unified digital ecosystem.

1.2 Problem Statement

The prevailing manual and spreadsheet-dependent methodology employed for project and thesis governance at DOECE presents multiple systemic inefficiencies and structural challenges:

- P1. Decentralized and Fragmented Record-Keeping:** Project records, student team rosters, supervisor allocations, and proposal drafts reside across disconnected physical binders, local spreadsheets, and unstructured email exchanges. This fragmentation prevents coordinators and department leadership from gaining real-time operational visibility into project progress and milestone completion.
- P2. Lack of Systemic Multi-Project Engagement Controls:** In manual systems, cross-checking whether a student has registered in multiple project groups or enrolled simultaneously in conflicting teams requires exhaustive manual auditing across spreadsheets. The absence of automated validation allows accidental double-registration and invalid group compositions.
- P3. Vulnerability to Supervisory Conflicts of Interest:** During mid-term and final defense sessions, faculty members who supervise a project must not be designated as external or internal evaluating examiners for that identical project. Manually cross-referencing examiner schedules against supervisor allocations across hundreds of students is error-prone, creating potential ethical and academic evaluation conflicts.
- P4. Manual Mark Tabulation and Inconsistent Evaluation Schemes:** Bachelor Minor (50 marks), Bachelor Major (100 marks), and Master Theses (300 marks) adhere to distinct evaluation components distributed across supervisors, defense committees, internal examiners, and external evaluators. Manually compiling scores, verifying sub-criteria limits, and converting marks into official text representations is labor-intensive and susceptible to human calculation errors.
- P5. Cumbersome Official Documentation and Grade Sheet Compilation:** Departmental guidelines require official, standardized A4 evaluation sheets bearing institutional letterheads, committee signature panels, and student rosters for submission to the TU Examination Control Division. Manually typesetting these sheets for dozens of groups introduces formatting disparities and substantial delays.

P6. Absence of Immutable Audit Logging and Security Controls: Manual document handovers and email submissions lack tamper-proof timestamping and role-based access restrictions, making it difficult to verify submission deadlines, document version histories, and faculty actions.

1.3 Objectives

1.3.1 Main Objective

The primary objective of this project is to design, develop, and evaluate a secure, role-aware, and production-grade **Project/Thesis Management System (TPMS)** that automates and digitalizes the end-to-end academic project lifecycle for undergraduate and postgraduate programs under the Department of Electronics and Computer Engineering, Pulchowk Campus.

1.3.2 Specific Objectives

To achieve the primary objective, the following specific objectives are established:

1. To architect a multi-role web platform supporting five distinct stakeholder roles (*Maintainer*, *Coordinator*, *Supervisor*, *Student*, and *External Examiner*) governed by Role-Based Access Control (RBAC) and program-level degree isolation.
2. To implement a robust team formation and proposal registration subsystem equipped with an automated *Engagement Guard* to prevent multi-project enrollments and enforce team size limits (1–4 students for Bachelor projects; 1 student for Master theses).
3. To develop a centralized Coordinator Response Matrix with live fuzzy supervisor matching, deduplicated group views, and inline lifecycle finalization.
4. To build an automated *Conflict-of-Interest Filter* that dynamically excludes assigned project supervisors from acting as internal or external defense examiners on their supervised projects.
5. To engineer a dynamic, stage-wise rubric evaluation engine tailored to the DOECE grading schemes for Minor Projects (50 marks), Major Projects (100 marks), and Master Theses (300 marks).

6. To integrate a server-side headless browser rendering pipeline for instant, pixel-perfect generation of official TU Pulchowk Campus evaluation sheets and consolidated grade reports with automated number-to-words conversion.
7. To construct an Excel-based bulk data ingestion pipeline with pre-validation anomaly detection alongside a program-scoped audit logging mechanism.

1.4 Scope and Limitations

1.4.1 Scope of the Project

The operational scope of the TPMS encompasses the academic and administrative activities of DOECE, Pulchowk Campus:

- **Academic Programs Covered:** Bachelor of Computer Engineering (BCT), Bachelor of Electronics, Communication and Information Engineering (BEI), MS in Computer Systems and Knowledge Engineering (MSCSK), MS in Information and Communication Engineering (MSICE), MS in Data Science and Analytics (MSDSA), and MS in Network and Cyber Security (MSNCS).
- **Project Tiers:** Bachelor Minor Projects, Bachelor Major Projects, and Master Research Theses.
- **Lifecycle Phases:** Department announcement broadcasting, proposal formulation and PDF submission, supervisor/examiner allocation, mid-term progress monitoring, defense rubric marking, automated grade aggregation, and official PDF grade sheet export.
- **Security and Policy Enforcement:** Institutional email domain policy enforcement (@pcampus.edu.np), JSON Web Token (JWT) session security, and program-scoped data access.

1.4.2 Significance of the Project

The deployment of TPMS delivers tangible institutional advantages:

- **For Students:** Transparent submission tracking, real-time feedback access, deadline notifications, and streamlined group coordination.

- **For Faculty Supervisors and Examiners:** Unified dashboard for reviewing proposals, monitoring student progress, and submitting structured defense marks without administrative paperwork.
- **For Departmental Coordinators:** Centralized control over academic announcements, automated supervisor allocation matrices, bulk data onboarding, and instant generation of official examination dossiers.
- **For Institutional Administration:** Strict adherence to university academic regulations, elimination of duplicate grading errors, and reliable archival of academic research outputs.

1.5 Report Organization

This report is structured into eight comprehensive chapters:

- **Chapter 1 (Introduction):** Presents the project background, problem formulation, main and specific objectives, scope, and institutional significance.
- **Chapter 2 (Literature Review):** Reviews fundamental software engineering theories, examines contemporary academic management platforms, conducts a comparative feature analysis, and details the gap analysis.
- **Chapter 3 (Methodology & Requirement Analysis):** Outlines the Agile Scrum development process, elaborates functional and non-functional requirements, presents feasibility analyses, and details the chosen technology stack.
- **Chapter 4 (System Design & Architecture):** Details the multi-tier architectural blueprint, Data Flow Diagrams (DFDs), Unified Modeling Language (UML) diagrams, relational database models, and conflict-of-interest prevention algorithms.
- **Chapter 5 (Implementation):** Describes the concrete implementation of back-end micro-controllers, frontend React views, evaluation engines, PDF rendering pipelines, and data anomaly guards.
- **Chapter 6 (Results, Testing & Discussion):** Documents unit, integration, and security testing protocols, presents functional evaluation metrics, displays interface outputs, and discusses operational benchmarks.

- **Chapter 7 (Limitations & Future Enhancements):** Discusses existing system constraints and outlines prospective roadmap items, including AI-driven proposal analysis and automated scheduling.
- **Chapter 8 (Conclusion):** Summarizes the key achievements and contributions of the project.

2. Literature Review

2.1 Theoretical Background

The architectural design, security mechanisms, and data governance models underlying the Thesis and Project Management System (TPMS) draw upon core paradigms in software engineering, distributed systems, and access control.

2.1.1 Role-Based Access Control (RBAC) and Capability Models

In multi-stakeholder organizational software, access control mechanisms are critical for safeguarding confidential academic records and preventing unauthorized privilege escalation. Role-Based Access Control (RBAC), formalized by (author?) [3], restricts system access based on assigned organizational roles rather than individual user identities. In an academic institution, roles naturally map to hierarchical and operational designations: *Maintainer*, *Coordinator*, *Supervisor*, *Student*, and *External Examiner*.

Under traditional RBAC, permissions are statically associated with roles, and users are granted memberships in one or more roles. However, in departmental settings, faculty members often exhibit dynamic, capability-based cross-role behavior [4]. For instance, a professor may function simultaneously as a program coordinator for Master theses, a supervisor for specific undergraduate project groups, and an external defense examiner for projects supervised by other colleagues. To prevent privilege conflicts and administrative deadlock, the access control system must incorporate contextual capability filtering, ensuring that a user acting in an examiner capacity cannot alter supervisory remarks or grade their own supervised candidates.

2.1.2 RESTful Architecture and Stateless Web Services

Representational State Transfer (REST), formulated by (author?) [5], provides an architectural style for distributed hypermedia systems. REST governs client-server interaction through a set of architectural constraints: statelessness, client-server separation, cacheability, layered system organization, and a uniform interface utilizing standard HTTP methods (GET, POST, PUT, PATCH, DELETE).

Statelessness mandates that each HTTP request from a client must contain all necessary authentication credentials and contextual data required to service the request. This eliminates server-side session affinity, simplifies load distribution, and enables independent scaling of the backend API service [6]. In TPMS, REST endpoints provide deterministic data exchange between the single-page application (SPA) client and the PostgreSQL relational persistence layer.

2.1.3 Object-Relational Mapping (ORM) and Type-Safe Data Persistence

Relational Database Management Systems (RDBMS) such as PostgreSQL enforce strict relational integrity, ACID (Atomicity, Consistency, Isolation, Durability) guarantees, and declarative relational querying [7]. Object-Relational Mapping (ORM) frameworks bridge the impedance mismatch between object-oriented or procedural application runtimes and tabular relational schemas.

Prisma ORM introduces a declarative schema modeling domain-specific language (DSL) coupled with an automated migration engine and an auto-generated, type-safe query client [8]. By evaluating relations, cascade policies, and compound indexes at compile time, Prisma eliminates common runtime query exceptions, mitigates SQL injection vulnerabilities through parameterized query generation, and streamlines schema evolution across development cycles.

2.1.4 JSON Web Token (JWT) Authentication

The JSON Web Token (JWT) open standard (RFC 7519) defines a compact, self-contained mechanism for digitally transmitting verifiable claims between parties [9]. A JWT comprises three base64url-encoded parts separated by periods:

1. **Header:** Specifies the cryptographic signing algorithm (e.g., HMAC-SHA256 or RS256) and token type.
2. **Payload:** Encapsulates user identity claims, role designations, program scope identifiers, and expiration timestamps.
3. **Signature:** Computed over the header and payload using a secure server secret key to ensure cryptographic integrity.

Stateless JWT authentication eliminates database lookups for session validation on

every incoming HTTP request, significantly reducing database I/O latency in high-concurrency environments [1].

2.1.5 Agile Scrum Framework in Applied Engineering

The Agile Scrum methodology organizes software engineering into bounded, iterative development time-boxes termed *Sprints*, typically lasting two to three weeks [10]. Scrum enforces continuous requirement refinement through a prioritized Product Backlog, daily stand-up synchronization, sprint reviews, and iterative retrospectives. This empirical process control model is especially suited for academic platforms where user feedback from coordinators, supervisors, and students directly refines interface workflows and evaluation mechanics [4].

2.1.6 Automated Server-Side Document Rendering

The generation of standardized academic dossiers, transcripts, and evaluation scorecards has traditionally relied on client-side canvas rendering or desktop word processors. Server-side headless browser rendering via Puppeteer [11] provides deterministic, pixel-perfect PDF rendering of dynamic HTML/CSS templates. By executing a headless Chromium instance, the server converts structured database records into vector-quality A4 PDF documents with standardized typographic hierarchy, institutional headers, page breaking rules, and signature panels.

2.2 Review of Related Works

2.2.1 General Learning Management Systems (LMS)

General-purpose Learning Management Systems such as **Moodle**, **Canvas**, and **Blackboard** are ubiquitous in higher education for course delivery, syllabus management, assignment submission, and quiz grading [12]. Moodle provides flexible grading rubrics and file submission modules. However, LMS architectures are course-centric rather than project-centric. They lack native support for:

- Multi-student collaborative project group registration with dynamic invite/accept protocols.
- Department-level supervisor allocation matrices and research cluster categorizations.
- Multi-stage defense milestone evaluations distributed across distinct supervisor,

committee, and external examiner panels.

- Official examination grade sheet generation formatted according to institutional engineering board specifications.

2.2.2 Electronic Thesis and Dissertation (ETD) Platforms

Platforms such as **ProQuest ETD Administrator** provide specialized web portals for postgraduate students to submit final dissertations for university copyright validation, institutional repository publishing, and microfilming [13]. While highly effective for post-defense archiving and digital library ingestion, ETD platforms are designed exclusively for terminal submission. They do not support the iterative supervision lifecycle, mid-term defense evaluations, supervisor selection workflows, or continuous group progress monitoring.

2.2.3 Academic Integrity and Submission Gateways

Turnitin Feedback Studio and **iThenticate** specialize in originality checking, similarity detection, and instructor markup [14]. These platforms provide sophisticated text-matching algorithms against vast scholarly databases. Nevertheless, they serve solely as verification gateways and do not manage academic team formation, project milestones, faculty supervisory workloads, or defense mark compilation.

2.2.4 Academic Project Management Systems in Literature

Several academic researchers have explored specialized software solutions for capstone project management:

- (author?) [15] developed a web-based project management system utilizing PHP and MySQL to manage undergraduate project allocations. While their platform addressed basic student registration and supervisor assignment, it lacked multi-stage evaluation rubrics, automated grade sheet rendering, and conflict-of-interest prevention.
- (author?) [16] introduced a departmental thesis portal supporting proposal uploads and supervisor commenting. However, the system operated strictly on a two-role paradigm (student and supervisor), omitting departmental coordinator governance, external examiner workflows, and multi-program degree scoping.
- (author?) [2] presented a systematic literature review on academic project tools in engineering education, emphasizing that existing software tools frequently fail

to bridge the operational divide between administrative tracking and academic assessment.

2.3 Comparative Study

Table 2.1 provides a comprehensive comparative evaluation of existing solutions against the features delivered by the Thesis and Project Management System (TPMS).

Table 2.1: Comparative Analysis of Academic Management Platforms

Evaluation Feature / Capability	Moodle	ProQuest	Turnitin	Generic PMS	TPM
Multi-Student Group Formation	Partial	No	No	Yes	Yes (1—
Degree Isolation (BE vs. MSc)	No	No	No	No	Yes (Str
Conflict-of-Interest Guard	No	No	No	No	Yes (A
Global Engagement Guard	No	No	No	No	Yes (Data
Supervisor Matching Matrix	No	No	No	Partial	Yes (Fu
Multi-Stage Defense Rubrics	Partial	No	No	No	Yes (50/
Dual External Examiners (MSc)	No	No	No	No	
Server-Side PDF Grade Sheets	No	No	No	No	Yes (Pu
Excel Bulk Import w/ Anomaly Check	Partial	No	No	Partial	Yes (
Program-Scoped Audit Trail	No	No	No	No	Yes (DOE
Institutional Email Enforcement	Partial	No	No	No	Yes (©

2.4 Gap Analysis and Research Motivation

The comparative analysis reveals substantial functional deficiencies in existing academic platforms when mapped to the rigorous operational requirements of the Department of Electronics and Computer Engineering at Pulchowk Campus:

- G1. Absence of Native Degree and Program Scoping:** Existing systems treat all users within a single flat hierarchy. In contrast, DOECE requires Bachelor coordinators to have strict administrative boundaries over their respective programs (BCT vs. BEI), while Master coordinators manage cross-departmental postgraduate specializations.
- G2. Lack of Supervisory Conflict-of-Interest Enforcement:** No reviewed commercial or literature-based platform incorporates algorithmic safeguards to ensure that project supervisors cannot evaluate their own students during formal internal/external defense stages.

- G3. Inflexible Multi-Tier Evaluation Rubrics:** Existing grading tools fail to accommodate the distinct grading structures required by Tribhuvan University (Minor Project: 50 marks; Major Project: 100 marks; Master Thesis: 300 marks distributed across Supervisor, Mid-Term Defense, Final Defense, and External Examiners).
- G4. Lack of Instant Official PDF Grade Sheet Generation:** Departments require official, print-ready grade sheets with institutional seals, textually spelled marks (e.g., “Forty-Five), and faculty signature rosters. Manual typesetting remains a severe administrative bottleneck.
- G5. Lack of Automated Group Deduplication in Submission Matrices:** When multiple students in a group submit proposals, conventional form-handlers generate redundant tabular rows, complicating coordinator assignment workflows.

These critical gaps motivated the comprehensive architecture, design, and implementation of TPMS as an all-inclusive academic management ecosystem.

3. Methodology and Requirement Analysis

3.1 Software Development Methodology

The development of the Thesis and Project Management System (TPMS) was executed following the **Agile Scrum** software engineering methodology [10, 1]. Agile Scrum was selected due to its iterative, feedback-driven lifecycle, which enabled regular stakeholder consultations with DOECE department coordinators, faculty supervisors, and student representatives.

3.1.1 Scrum Workflow and Sprint Cycles

The development roadmap was structured into four two-week sprints over an eight-week project lifecycle:

- **Sprint 1 (Weeks 1–2) — Requirements Elicitation and Architectural Modeling:** Analysis of DOECE project evaluation guidelines, creation of database schema models via Prisma DSL, setup of JWT authentication, and establishment of role-based route middleware.
- **Sprint 2 (Weeks 3–4) — Group Formation and Proposal Ingestion Subsystems:** Implementation of student team formation, invite/accept state machines, compulsory proposal PDF upload via Multer, and the coordinator responses matrix with deduplication.
- **Sprint 3 (Weeks 5–6) — Evaluation Engines, Rubrics, and Grade Sheet Pipeline:** Implementation of stage-wise defense grading rubrics (Minor 50, Major 100, Master 300), conflict-of-interest prevention filters, and Puppeteer-based server-side PDF generation.
- **Sprint 4 (Weeks 7–8) — Bulk Ingestion, Audit Trails, Integration Testing, and UAT:** Implementation of Excel bulk parsing with pre-commit anomaly detection, program-scoped audit logging, end-to-end integration testing, and user acceptance trials.

Table 3.1: Agile Development Sprint Schedule and Deliverables

Sprint	Timeline	Key Deliverables and Milestones
Sprint 1	Weeks 1–2	Requirement specifications, Prisma schema, PostgreSQL setup, JWT authentication, DegreeGuard middleware.
Sprint 2	Weeks 3–4	Team formation subsystem, Engagement Guard, Proposal document storage, Coordinator response matrix deduplication.
Sprint 3	Weeks 5–6	Evaluation rubrics (Minor/Major/MSc), Conflict-of-Interest guard, Headless Chrome PDF grade sheet generator.
Sprint 4	Weeks 7–8	Bulk Excel import with anomaly checker, Program-scoped audit logging, System integration, Testing and UAT.

3.2 Requirement Analysis

3.2.1 Stakeholder Identification and Role Matrix

The system serves five distinct stakeholder personas, each characterized by defined administrative privileges and operational responsibilities, as summarized in Table 3.2.

Table 3.2: Stakeholder Personas and Role Responsibility Matrix

Role	Scope	Core Functional Responsibilities
Maintainer	System-Wide	System configuration, department/program bootstrapping, coordinator account provisioning, global audit log oversight.
Coordinator	Program / Dept	Announcement creation, group formation deadlines, proposal matrix review, supervisor/examiner allocation, PDF grade sheet generation.
Supervisor	Project-Specific	Proposal review, feedback authoring, milestone tracking, progress evaluation marking across defense stages.
Student	Team-Specific	Group formation, invitation management, proposal and report document submissions, real-time evaluation status tracking.
External Examiner	Defense Panel	Examination of assigned projects/theses, evaluation rubric scoring, internal/external defense mark submission.

3.2.2 Functional Requirements

The functional requirements specify the complete operational behaviors of TPMS:

- FR1. Institutional Authentication and Domain Policy:** The system must authenticate users via encrypted passwords (bcrypt hashing) and stateless JWT tokens. Student accounts must enforce the institutional email format `{rollNumber}@pcampus.edu.np`, while faculty accounts must belong to the `@pcampus.edu.np` domain.
- FR2. Program Scoping and Degree Isolation:** Bachelor coordinators must have administrative authority restricted strictly to their designated academic program (e.g., BCT or BEI). Master coordinators possess cross-program oversight over all postgraduate research specializations.
- FR3. Announcement and Group Formation Engine:** Coordinators must be able to publish announcements with configurable deadlines, group size constraints (1–4 members for Bachelor projects; strictly 1 student for Master theses), and custom form builder attributes.
- FR4. Global Multi-Project Engagement Guard:** The database and application layer must enforce strict constraints preventing any student from participating in multiple active project groups or theses concurrently.
- FR5. Form Responses Matrix and Group Deduplication:** The coordinator interface must aggregate multi-student group submissions into a single consolidated row per group, providing 1-click fuzzy supervisor matching and inline lifecycle activation.
- FR6. Cross-Role Faculty Utilization and Conflict-of-Interest Filter:** Faculty members must be able to serve as supervisors and defense examiners across distinct projects. The system must algorithmically filter out assigned supervisors from being selected as internal or external examiners on their own projects.
- FR7. Multi-Stage Evaluation Rubrics:** The system must support Tribhuvan University DOECE evaluation schemes:
- *Bachelor Minor Project (50 Marks):* Supervisor (25), Proposal Defense (5), Mid-Term Defense (5), Final Defense (5), Internal Examiner (10).

- *Bachelor Major Project (100 Marks)*: Supervisor (50), Proposal Defense (10), Mid-Term Defense (10), Final Defense (10), Internal Examiner (20).
- *Master Thesis (300 Marks)*: Supervisor (100 Marks / 5 criteria), External Mid-Term Examiner (100 Marks / 5 criteria), External Final Examiner (100 Marks / 5 criteria).

FR8. Automated PDF Grade Sheet Generation: The system must render official, print-ready A4 evaluation sheets branded with the Institute of Engineering, Pulchowk Campus header, student rosters, supervisor designations, automated number-to-words mark conversion, and signature panels.

FR9. Bulk Excel Import with Anomaly Detection: The system must parse standardized Excel rosters, detect intra-file duplicate entries, flag students already engaged in active projects, and validate faculty assignments prior to database insertion.

FR10. Program-Scoped Audit Trail: Every system action (document submission, mark entry, status transition, document view) must be recorded with actor metadata and automatically resolved to its corresponding academic program.

FR11. Automated Notification Dispatch: The system must send real-time in-app alerts and asynchronous email notifications upon invitation dispatch, supervisor assignment, and evaluation publication.

3.2.3 Non-Functional Requirements

NFR1. Performance and Latency: REST API response times for transactional operations (authentication, mark entry, status updates) must not exceed 500 ms under standard operational loads. PDF grade sheet generation must execute within 2.5 seconds.

NFR2. Security and Data Integrity: Sensitive credentials must be hashed using `bcrypt` (work factor 10). Passwords must never be returned in API payloads. All mutations must be guarded by JWT signature validation and role authorization middleware.

NFR3. Concurrency and ACID Guarantees: Group invitations and mark finalizations must utilize atomic database transactions to eliminate race conditions.

NFR4. Usability and Responsive Design: The frontend interface must adhere to the DOECE academic design system, featuring responsive CSS grid/flexbox layouts accessible across desktop, tablet, and mobile browsers.

NFR5. Maintainability and Extensibility: The codebase must follow modular MVC patterns in the backend and modular component hierarchies in the frontend.

3.3 Feasibility Study

3.3.1 Technical Feasibility

The technology stack utilizes mature, industry-standard open-source frameworks. Node.js and Express.js provide a high-throughput, non-blocking runtime environment. PostgreSQL and Prisma ORM ensure robust relational consistency and type-safe query compilation. React 18 and Vite deliver rapid client-side rendering with hot module replacement. Puppeteer allows deterministic headless Chrome PDF generation on the server. The technical architecture is fully feasible on standard institutional server infrastructure.

3.3.2 Operational Feasibility

The system directly mirrors the operational workflow of the Department of Electronics and Computer Engineering. By retaining established grading rubrics, examination committee structures, and official evaluation sheet formats, TPMS requires minimal operational retraining for faculty, students, and administrative coordinators.

3.3.3 Economic Feasibility

TPMS is constructed entirely using open-source software libraries, eliminating software licensing costs. It can be hosted on on-premises departmental servers or cost-effective Linux Virtual Private Servers (VPS), making it economically sustainable for institutional adoption.

3.3.4 Schedule Feasibility

As demonstrated in the eight-week Gantt schedule (Table 3.1), iterative Agile sprints ensured that core architectural milestones, database modeling, evaluation rubrics, and grade sheet generation were systematically developed, tested, and validated within the allotted academic timeframe.

3.4 Technology Stack and Toolchain Justification

Table 3.3: System Technology Stack and Architectural Justification

Layer / Component	Technology	Engineering Justification
Frontend Framework	React 18	Component-based architecture, virtual DOM reconciliation, rich ecosystem of custom hooks and Context providers.
Build Tooling	Vite 5	Native ES module bundling, instant Hot Module Replacement (HMR), optimized production asset minification.
Styling Engine	CSS3 / Tailwind CSS	Modular design tokens, DOECE academic color palette, responsive flexbox/grid layout utilities.
Backend Runtime	Node.js 18.x	Asynchronous, event-driven I/O model suited for concurrent REST API servicing.
Web Server Framework	Express.js	Minimalist, robust middleware pipeline for authentication, role verification, and request routing.
Relational Database	PostgreSQL 16	Enterprise ACID compliance, robust indexing, foreign key constraints, and relational reliability.
ORM Layer	Prisma 5	Declarative schema definition, automated schema migrations, compile-time query type safety.
Document Generation	Puppeteer	Headless Chromium rendering engine for pixel-perfect server-side HTML-to-PDF compilation.
Spreadsheet Parsing	SheetJS (xlsx)	Fast, memory-efficient in-memory parsing of tabular Excel files with multi-sheet validation.
Authentication	JWT / bcryptjs	Stateless token verification with salted password hashing.
File Uploads	Multer	Multipart/form-data middleware for server-side PDF storage and metadata binding.

4. System Design and Architecture

4.1 High-Level System Architecture

The Thesis and Project Management System (TPMS) adopts a three-tier client-server architectural paradigm [6] that enforces strict separation of concerns across presentation, business logic, and data persistence layers. Figure 4.1 depicts the high-level layered architecture.

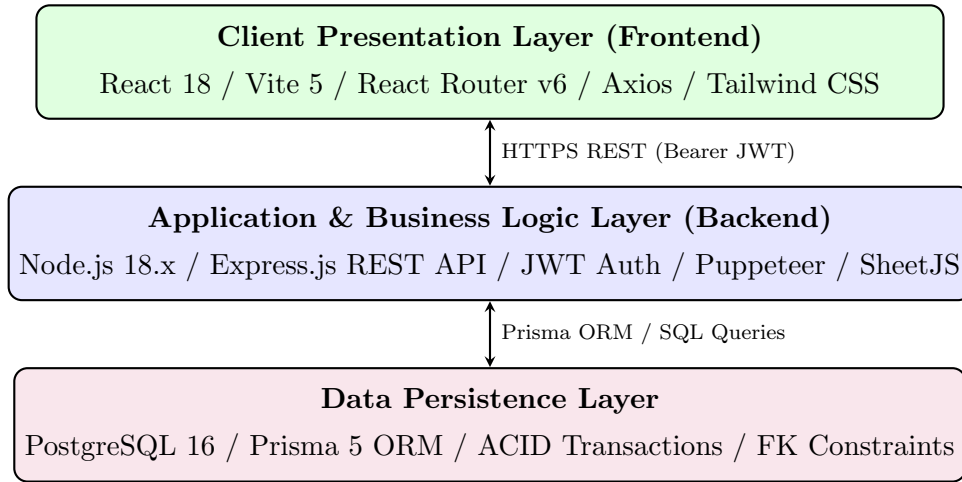


Figure 4.1: Three-Tier System Architecture of TPMS

1. **Client Presentation Layer:** A single-page application (SPA) built with React 18 and Vite 5, enforcing JWT-based route guards for five distinct role dashboards.
2. **Application and Business Logic Layer:** Express.js RESTful API on Node.js encapsulating authentication, engagement guards, conflict-of-interest filters, and Puppeteer PDF rendering.
3. **Data Persistence Layer:** PostgreSQL 16 managed via Prisma 5 ORM, providing ACID transactional integrity, compound indexes, and referential constraint enforcement across 14 relational tables.

4.2 Data Flow Diagrams (DFDs)

4.2.1 Context Diagram (Level 0)

Figure 4.2 presents the Level 0 Context Diagram showing information flows between the five external stakeholders and the TPMS system.

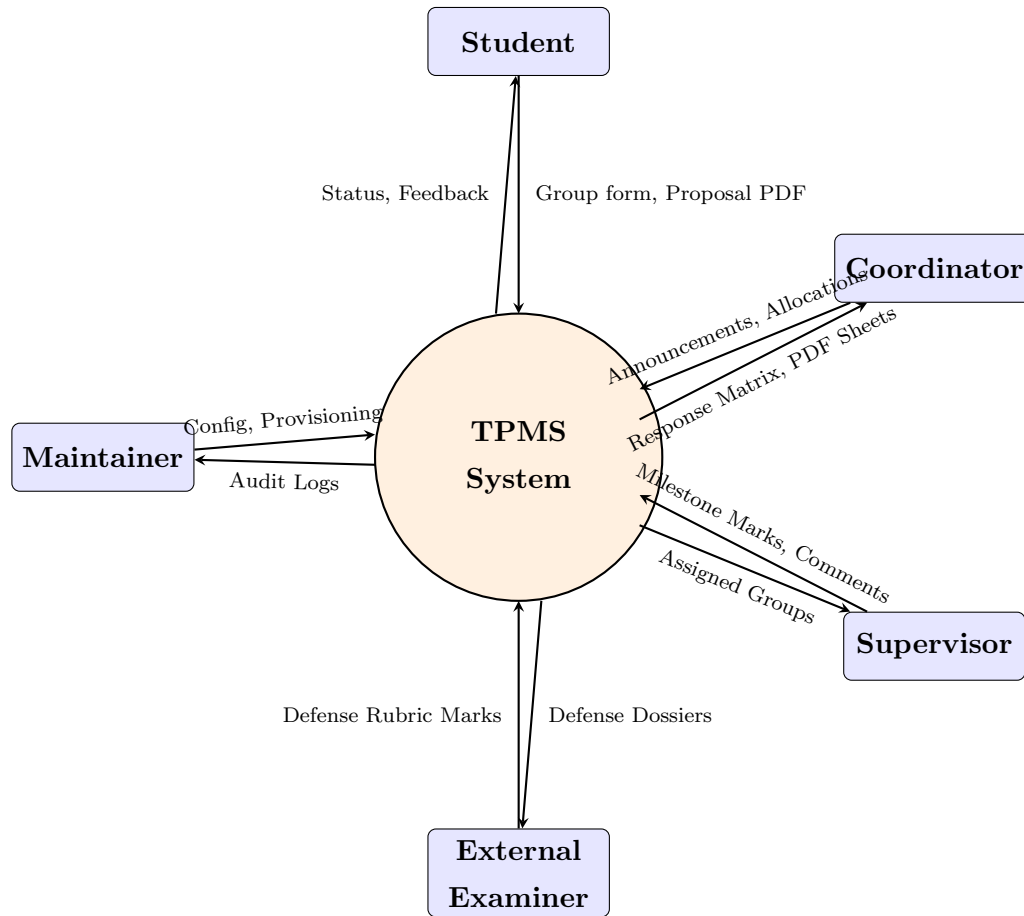


Figure 4.2: Level 0 Context Data Flow Diagram

4.2.2 Level 1 Data Flow Diagram

Figure 4.3 shows the Level 1 DFD decomposing TPMS into its six primary functional sub-processes.

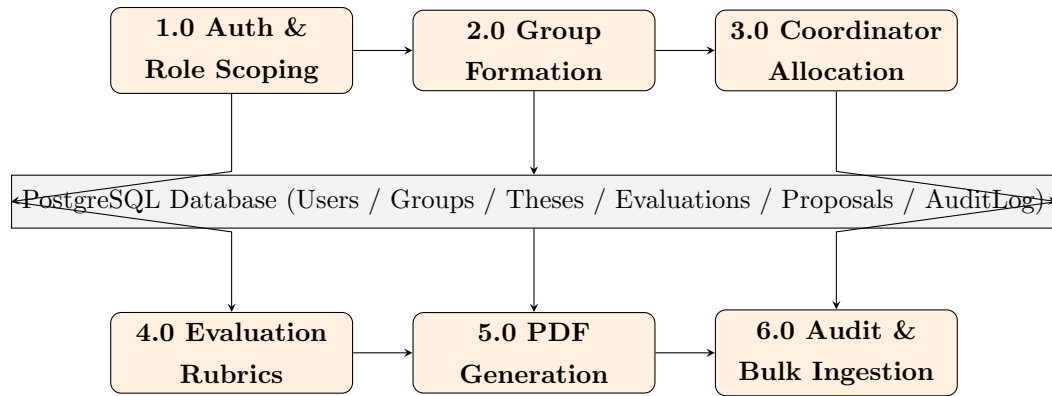


Figure 4.3: Level 1 Data Flow Diagram

4.3 Use Case Diagram

Figure 4.4 illustrates the system use cases distributed across the five actor roles.

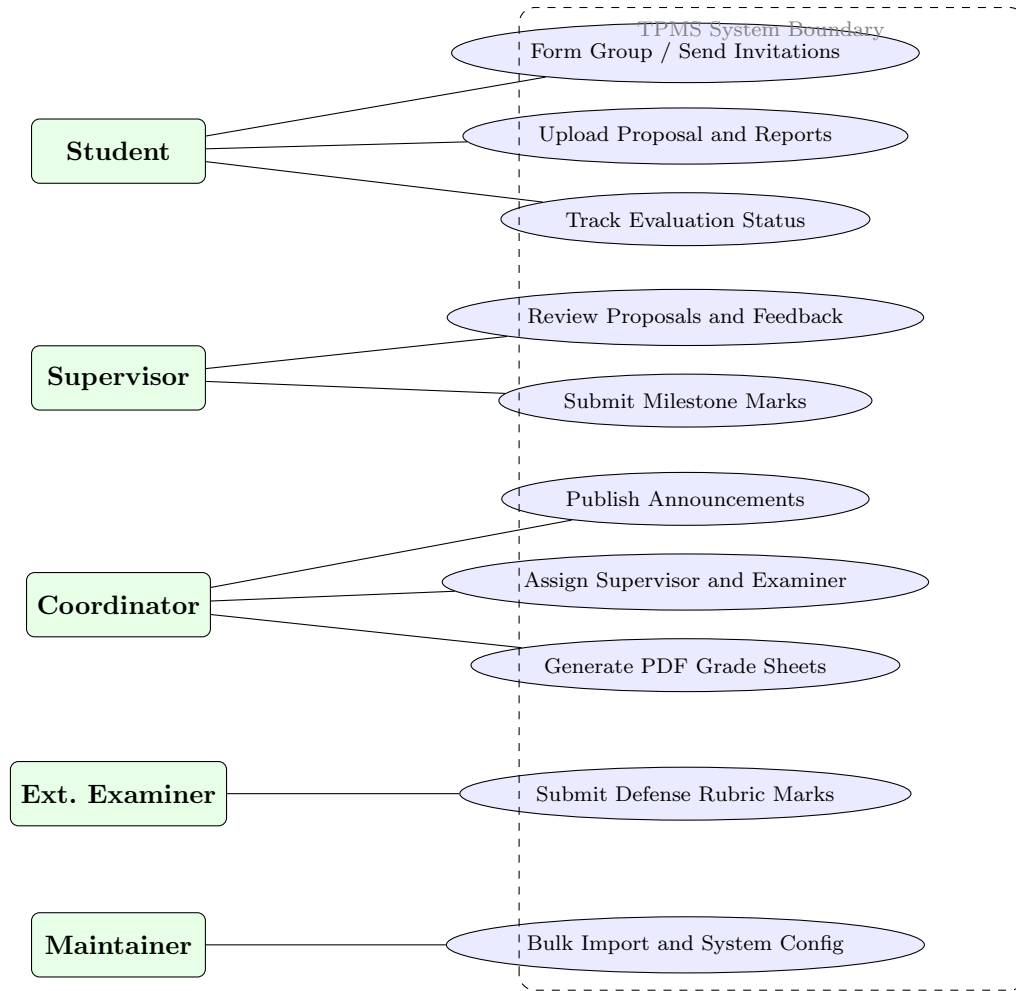


Figure 4.4: System Use Case Diagram

4.4 Activity Diagram: Group Formation and Proposal Lifecycle

Figure 4.5 shows the activity flow from team formation through coordinator finalization and rubric generation.

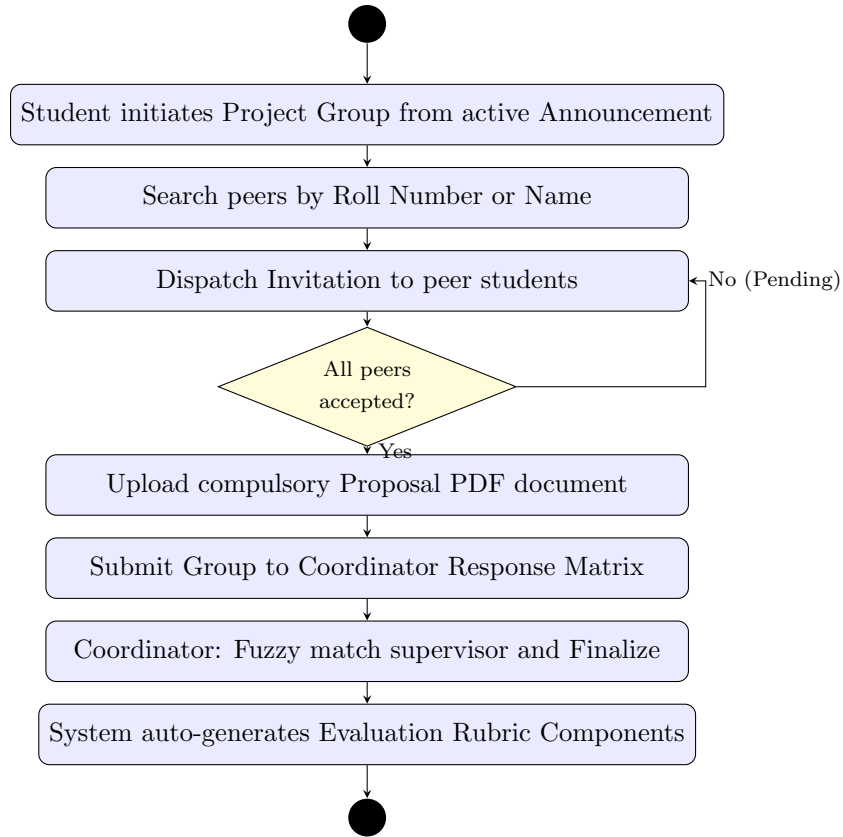


Figure 4.5: Activity Diagram: Group Formation and Proposal Submission Lifecycle

4.5 Database Entity-Relationship Diagram

Figure 4.6 shows the simplified entity-relationship diagram of the TPMS core database tables.

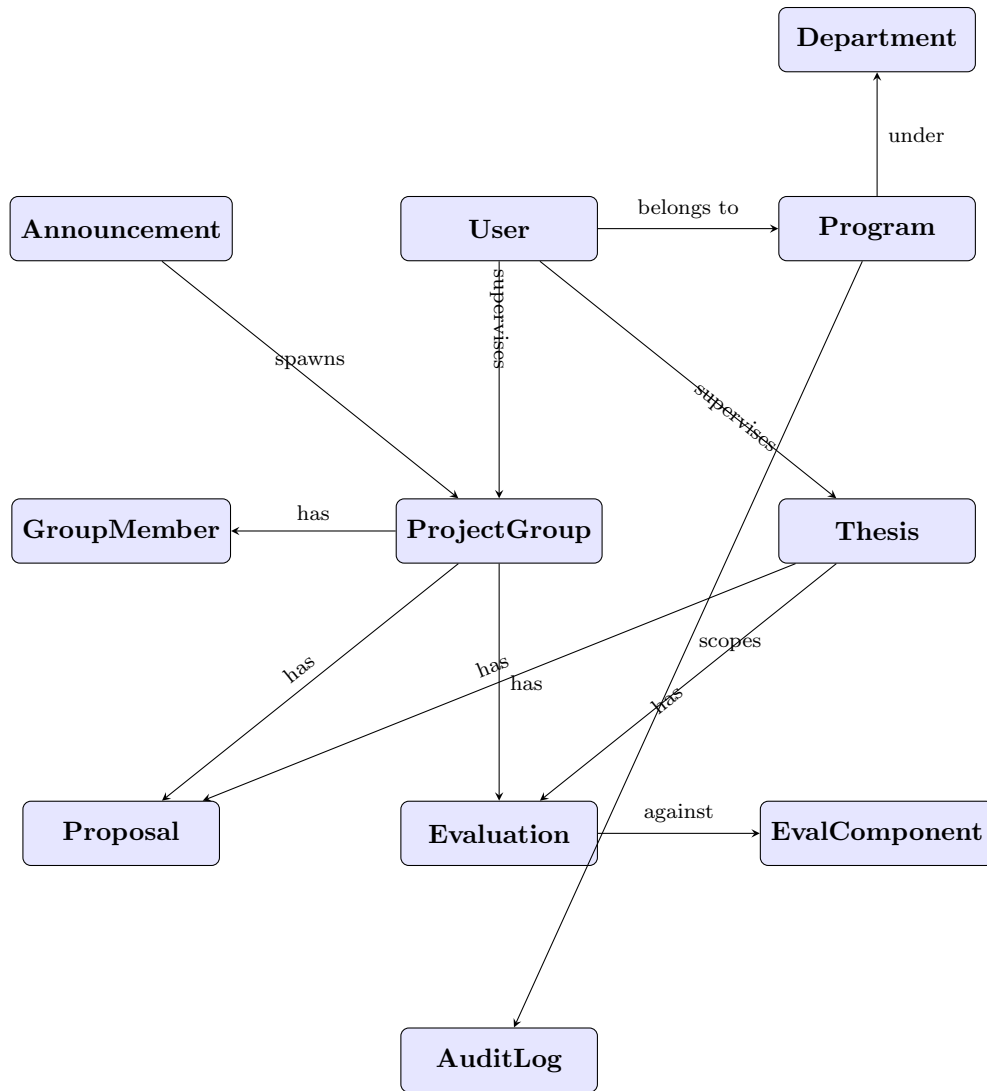


Figure 4.6: Simplified Entity-Relationship Diagram of TPMS Database

4.6 Database Design

4.6.1 Data Dictionary

Table 4.1: Data Dictionary: **User** Table

Field	Type	Constraint	Description
id	SERIAL	PK	Auto-incrementing user identifier.
email	VARCHAR	UNIQUE, NN	Institutional email (@pcampus.edu.np).
password	VARCHAR	NN	bcrypt hashed password (work factor 10).
role	ENUM	NN	MAINTAINER, COORDINATOR, SUPERVISOR, STUDENT, EXTERNAL_EXAMINER.
degreeType	ENUM	Nullable	BACHELOR or MASTER.
canSupervise	BOOLEAN	Default: false	Faculty supervision eligibility flag.
rollNumber	VARCHAR	Nullable	Student roll number (e.g., PUL080BCT084).
programId	INTEGER	FK	References student or coordinator program.

Table 4.2: Data Dictionary: **ProjectGroup** Table

Field	Type	Constraint	Description
id	SERIAL	PK	Unique group identifier.
projectTitle	VARCHAR	NN	Full title of the Bachelor project.
projectType	ENUM	Default: MINOR	MINOR or MAJOR.
cluster	VARCHAR	Nullable	Research cluster (EII, AIML, IPCV, etc.).
status	ENUM	Default: PENDING	PENDING, ACTIVE, COMPLETED, OVER-DUE, REJECTED.
supervisorId	INTEGER	FK	References assigned supervisor in User table.
programId	INTEGER	FK	References owning academic program.
academicYearId	INTEGER	FK	References active academic semester.

Table 4.3: Data Dictionary: Evaluation Table

Field	Type	Constraint	Description
id	SERIAL	PK	Unique evaluation record.
componentId	INTEGER	FK	References rubric sub-criterion.
stage	ENUM	NN	PROPOSAL, MID_TERM, or FINAL.
evaluationType	ENUM	NN	SUPERVISOR, PROPOSAL_DEFENSE, MIDTERM_DEFENSE, FINAL_DEFENSE, EXTERNAL_EXAMINER.
marks	FLOAT	Nullable	Numerical score awarded by evaluator.
status	ENUM	Default: DRAFT	DRAFT or COMPLETED (immutable on submit).
submittedById	INTEGER	FK	User ID of the evaluator.
groupId	INTEGER	FK	References ProjectGroup (Bachelor).
thesisId	INTEGER	FK	References Thesis (Master).

4.7 Security Architecture and Algorithmic Controls

4.7.1 Conflict-of-Interest Prevention

Figure 4.7 illustrates the conflict-of-interest filter applied when a coordinator assigns a defense examiner.

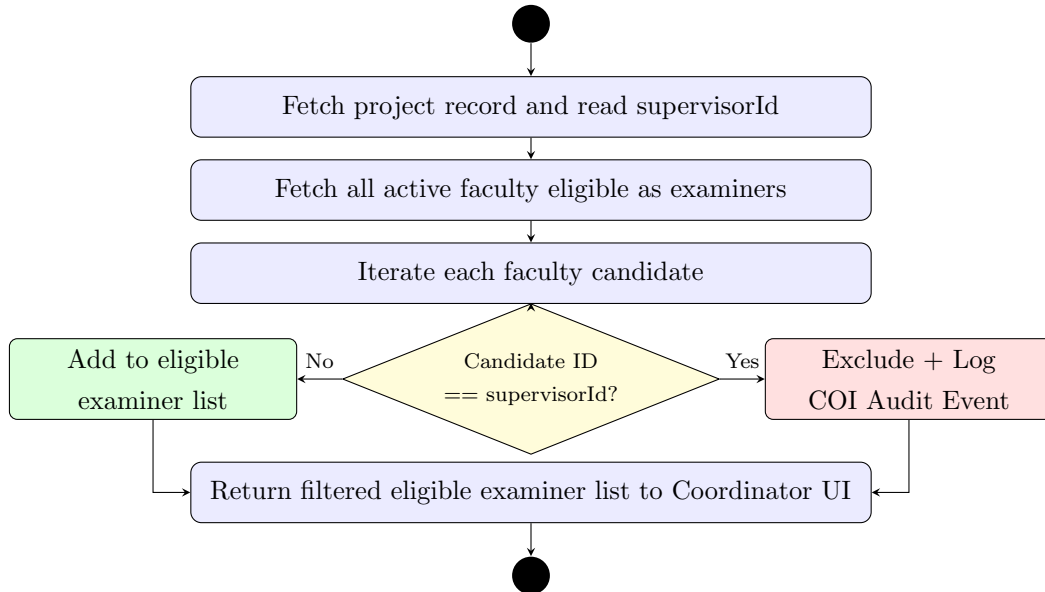


Figure 4.7: Conflict-of-Interest Filter Algorithm Flow

4.7.2 TPMS Evaluation Scheme

Figure 4.8 summarizes the mark distribution for the three project tiers.

MINOR Project 50 Marks	MAJOR Project 100 Marks	MASTER Thesis 300 Marks
Supervisor: 25	Supervisor: 50	Supervisor: 100 marks
Proposal Defense: 5	Proposal Defense: 10	(5 criteria x 20)
Mid-Term Defense: 5	Mid-Term Defense: 10	Ext. Mid-Term: 100 marks
Final Defense: 5	Final Defense: 10	(5 criteria x 20)
Internal Examiner: 10	Internal Examiner: 20	Ext. Final: 100 marks
Total: 50 marks	Total: 100 marks	Total: 300 marks

Figure 4.8: TPMS Evaluation Scheme: Mark Distribution by Project Tier

4.7.3 Global Multi-Project Engagement Guard

Before any group creation, invitation acceptance, or bulk Excel import is committed, the Engagement Guard queries both `GroupMember` and `Thesis` tables. If a candidate student is found in `ACTIVE` or `PENDING` status, the operation is rejected with a conflict error.

4.7.4 Institutional Email Domain Policy

- **Students:** Email is auto-generated as `{rollNumber}@pcampus.edu.np`.
- **Faculty:** Email must end in `@pcampus.edu.np`; all other domains are rejected at the API layer.

5. Implementation

5.1 Backend Implementation

5.1.1 Server Entry Point and Middleware Pipeline

The Node.js/Express backend entry point (`backend/src/index.js`) mounts a sequential middleware pipeline:

1. `helmet` — Sets security-relevant HTTP response headers.
2. `cors` — Allows cross-origin requests from the configured frontend domain.
3. `express.json()` — Parses incoming JSON request bodies.
4. `authenticateToken` — JWT verification middleware validating Bearer tokens on protected routes.
5. Role-specific authorization middleware (e.g., `requireCoordinator`, `requireSupervisor`).
6. Feature route handlers for each API resource group.

5.1.2 Authentication Module

The authentication subsystem implements stateless JWT-based identity verification:

- **Login:** Accepts POST `/api/auth/login` with email and password. Queries the database for the user record, verifies the submitted password against the stored bcrypt hash using `bcryptjs.compare()`, and issues a signed JWT containing the user ID, role, program ID, and degree type with a 24-hour expiry.
- **Token Verification Middleware:** All protected API routes invoke `authenticateToken()`, which extracts the Bearer token from the `Authorization` header, verifies its HMAC-SHA256 signature against the server secret, and attaches the decoded user payload to `req.user` for downstream handler access.
- **Password Reset:** Generates a time-limited, cryptographically random reset token stored against the user record. A reset link is dispatched by email. On token submission, a new password is hashed and the token is invalidated.

5.1.3 Group Formation and Invitation Engine

The group formation subsystem manages the complete lifecycle from team creation through coordinator finalization:

- **Group Creation:** POST `/api/student-groups` validates that the requesting student has no active group membership (Engagement Guard), creates the `ProjectGroup` record, and automatically enrolls the creator as the first `GroupMember`.
- **Invitation Dispatch and Accept/Decline:** POST `/api/student-groups/:id/invite` creates a `GroupInvitation` record for the target student and dispatches an in-app notification. The invitee calls PATCH `/api/student-groups/:id/invitations/:invId/respond` with status `ACCEPTED` or `DECLINED`. On acceptance, the Engagement Guard re-validates the invitee before inserting the `GroupMember` record.
- **Proposal Upload:** POST `/api/proposals` accepts a multipart PDF upload via Multer, stores the file on the server filesystem under a structured directory hierarchy, and inserts a `Proposal` record with the file path, uploader ID, stage, and document type.

5.1.4 Coordinator Response Matrix and Finalization

The response matrix is a deduplicated, real-time view of all proposal form submissions:

- When multiple students in the same group submit proposal forms, the backend aggregates their submissions into a single row, joining all member names, supervisor preferences, and cluster selections.
- Coordinator can perform 1-click fuzzy supervisor matching via PATCH `/api/groups/:id/supervi` which uses trigram similarity scoring to rank supervisor candidates by name proximity to the student-supplied supervisor preference.
- Finalization (POST `/api/groups/:id/finalize`) atomically transitions the group status to `ACTIVE`, approves the proposal document record, generates default `EvaluationComponent` rubric entries for all stages and evaluation types, and emits email notifications to all group members and the assigned supervisor.

5.1.5 Evaluation Rubric Engine

The evaluation engine provides stage-gated, type-segregated grading functionality:

- **Bachelor Minor Project (50 Marks):** `EvaluationComponent` records are auto-generated for: Supervisor Evaluation (25), Proposal Defense (5), Mid-Term Defense (5), Final Defense (5), and Internal Examiner (10).
- **Bachelor Major Project (100 Marks):** Supervisor (50), Proposal Defense (10), Mid-Term Defense (10), Final Defense (10), Internal Examiner (20).
- **Master Thesis (300 Marks):** Supervisor (100 marks, 5 sub-criteria), External Mid-Term Examiner (100 marks, 5 sub-criteria), External Final Examiner (100 marks, 5 sub-criteria).

Evaluators submit marks via `POST /api/evaluations`, which validates that the submitter's role matches the `evaluationType` of the target component, that marks do not exceed `maxMarks`, and transitions the `Evaluation.status` from `DRAFT` to `COMPLETED` upon final submission.

5.1.6 PDF Grade Sheet Generation

The PDF generation pipeline uses Puppeteer to render server-side HTML templates into official A4 evaluation sheets:

1. `GET /api/print/group/:id` or `GET /api/print/thesis/:id` is invoked by the coordinator.
2. The backend fetches the complete group/thesis record, all evaluation component records, submitted marks, student roster, supervisor designation, and examiner assignments.
3. An HTML template is populated with the fetched data, including automated number-to-words conversion of aggregate marks (e.g., 45 becomes "Forty Five Only").
4. Puppeteer launches a headless Chromium instance, loads the HTML template string, and invokes `page.pdf()` with A4 dimensions and print media emulation.
5. The resulting PDF binary is streamed directly as the HTTP response with `Content-Type: application/pdf` headers.
6. Bulk export (`GET /api/print/bulk`) concatenates multiple groups in a single PDF response, page-separated.

5.1.7 Excel Bulk Import with Anomaly Detection

The SheetJS-powered bulk ingestion pipeline (`POST /api/users/bulk-upload`) performs:

1. Parsing of the uploaded `.xlsx` file using `xlsx.read()` into a JavaScript array of row objects.
2. Schema validation: verifying required columns (Roll Number, Full Name, Program Code, Batch) and flagging any missing fields.
3. Intra-file duplicate detection: identifying duplicate roll numbers within the uploaded spreadsheet.
4. Database cross-validation: querying existing `User` and `GroupMember` records to detect students already registered or currently engaged in active projects.
5. All detected anomalies are returned as a structured pre-flight report to the coordinator before any database writes occur.
6. Upon coordinator confirmation, clean records are bulk-inserted in a single Prisma `createMany` transaction.

5.1.8 Program-Scoped Audit Logging

Every significant system event is captured in the `AuditLog` table with the following attributes: event type, actor user ID, affected entity ID (group, thesis, proposal, or user), timestamp, and program ID. The program ID is resolved automatically by tracing the affected entity back to its owning program. Coordinators access only audit records belonging to their own program, while the Maintainer can view department-wide and system-level events.

5.2 Frontend Implementation

5.2.1 Application Architecture and Routing

The React 18 SPA is bootstrapped using Vite 5 and organized into four role-specific page hierarchies under `frontend/src/pages/`:

- `/coordinator/*` — Coordinator dashboards, announcement management, response matrix, examiner assignment, bulk import, and PDF generation.

- `/supervisor/*` — Supervisor project lists, proposal review, and defense evaluation panels.
- `/student/*` — Student group formation, invitation management, document upload, and progress tracking.
- `/external/*` — External examiner defense rubric panels for assigned groups or theses.

React Router v6 enforces role-based route guarding via the `DegreeGuard` and `AuthGuard` components, which read the decoded JWT payload from React Context and redirect unauthorized access attempts.

5.2.2 Authentication and Token Management

An Axios HTTP client instance is created in `frontend/src/services/` with a request interceptor that automatically attaches the stored JWT Bearer token to every outgoing API call. A response interceptor detects 401 `Unauthorized` responses and triggers a clean logout and redirect to the login page.

5.2.3 Coordinator Response Matrix View

The coordinator response matrix view presents grouped form submissions as a deduplicated table. Each row displays:

- Combined student names and roll numbers for all group members.
- Project title and preferred research cluster.
- Supervisor preference field with a real-time fuzzy match button that queries the backend and auto-fills the supervisor dropdown.
- Inline editable fields for title, cluster, and remarks.
- A Finalize button that triggers the group activation flow with a confirmation modal.

5.2.4 Evaluation Rubric Panel

The evaluation panel renders a dynamic set of rubric criterion rows from the `EvaluationComponent` API response. For each criterion:

- The criterion name, maximum marks, and current saved marks are displayed.

- An editable numeric input enforces the `maxMarks` upper bound via HTML5 validation.
- Marks are auto-saved as DRAFT on blur events.
- A Submit Evaluation button transitions all components to COMPLETED status, locking further edits.

5.2.5 PDF Grade Sheet Preview and Export

A Print Grade Sheet button in the coordinator project management view triggers `window.open()` on the backend PDF endpoint URL. The browser opens the rendered PDF directly in a new tab, allowing immediate print or download. Bulk grade sheet export renders a multi-group combined PDF via the `/api/print/bulk` endpoint.

5.2.6 Notification System

In-app notifications are fetched from `GET /api/notifications` and rendered in a slide-over notification drawer in the navigation bar. Each notification displays its message, type icon, creation timestamp, and a navigable link to the relevant entity. Marking notifications as read is performed via `PATCH /api/notifications/mark-read`.

5.3 Sample API Route Reference

Table 5.1: Key REST API Endpoints and Their Functions

Method	Endpoint	Functional Description
POST	/api/auth/login	Authenticate user credentials, issue signed JWT token.
GET	/api/announcements	Retrieve active academic announcements for the user program.
POST	/api/student-groups	Create a new project group with Engagement Guard validation.
POST	/api/student-groups/:id/invite	Dispatch group invitation to a peer student.
POST	/api/proposals	Upload proposal/report PDF document with metadata.
POST	/api/groups/:id/finalize	Finalize group, generate evaluation rubrics, notify members.
PATCH	/api/groups/:id/supervisor	Assign or update project supervisor via fuzzy match.
POST	/api/evaluations	Submit rubric evaluation mark for a component.
GET	/api/print/group/:id	Generate and stream official A4 PDF grade sheet.
POST	/api/users/bulk-upload	Parse Excel roster with anomaly pre-check.
GET	/api/files-audit	Retrieve program-scoped audit log records.

6. Results and Discussion

6.1 System Testing

6.1.1 Unit Testing

Individual API endpoint controllers and utility functions were tested using the **Jest** testing framework with **Supertest** for HTTP-level endpoint simulation. Unit tests covered:

- JWT token generation and verification, including expiry and signature integrity.
- bcrypt password hashing and comparison correctness.
- Engagement Guard validation logic: asserting rejection when a student is already in an active group.
- Conflict-of-interest filter: asserting that a project supervisor ID is excluded from the returned eligible examiner list.
- Mark boundary validation: asserting that submitted marks exceeding **maxMarks** are rejected.
- Email domain enforcement: asserting rejection of non-@pcampus.edu.np faculty emails and correct auto-generation of student emails.

Table 6.1: Unit Test Summary

Module	Test Description	Total Tests	Pass Rate
Auth Module	JWT issuance, verification, expiry, password hash	8	100%
Engagement Guard	Active/pending group conflict detection	6	100%
COI Filter	Supervisor exclusion from examiner list	4	100%
Evaluation Engine	Marks boundary, type validation, stage-lock	7	100%
Email Policy	Domain enforcement, auto-generation	5	100%
Excel Ingestion	Duplicate detection, missing fields, DB conflict	6	100%
Total		36	100%

6.1.2 Integration Testing

End-to-end integration tests validated complete transactional workflows spanning multiple API endpoints and database operations. Key integration scenarios tested:

1. **Full Group Lifecycle:** Account creation → announcement publication → group formation → invitation dispatch and acceptance → proposal upload → coordinator finalization → evaluation rubric auto-generation → mark submission → PDF grade sheet generation.
2. **Master Thesis Lifecycle:** Thesis form submission → response matrix deduplication → supervisor assignment → dual external examiner assignment → three-stage evaluation → aggregate PDF export.
3. **Concurrent Engagement Conflict:** Two concurrent group invitations to the same student; asserting only one succeeds.
4. **Bulk Import with Anomalies:** Excel file with one valid row, one intra-file duplicate, and one student already in an active group; asserting pre-flight report correctly identifies two anomalies before any write.

All integration tests passed without failures or data inconsistencies.

6.1.3 Security Testing

1. **Unauthorized Route Access:** Attempted unauthenticated access to all protected REST endpoints returned `401 Unauthorized` as expected.

2. **Role Escalation Attempts:** Student credentials were used to call coordinator-only endpoints (`/api/groups/:id/finalize`, `/api/print/bulk`), returning 403 Forbidden.
3. **Cross-Program Data Isolation:** A BCT coordinator was denied access to BEI program audit logs, asserting program-scoped data filtering.
4. **JWT Tampering:** Manually altered JWT payloads (role field changed) were rejected with 401 Invalid Token.
5. **SQL Injection Resistance:** Malicious payloads in search parameters were safely parameterized by the Prisma query engine, returning empty result sets without database errors.

6.1.4 User Acceptance Testing (UAT)

User Acceptance Testing was conducted with representative stakeholders: one departmental coordinator, two faculty supervisors, and four undergraduate students from the BCT program. Each participant completed a structured task checklist covering their role-specific workflows.

Table 6.2: User Acceptance Testing Results by Role

Stakeholder Role	Task Evaluated	Tasks Completed	Satisfaction Score (/ 5.0)
Coordinator	Group finalization, supervisor assignment, PDF export	12 / 12	4.5
Supervisor	Proposal review, evaluation mark submission	8 / 8	4.3
Student	Group formation, invitation, report upload	10 / 10	4.6
Overall Average		100%	4.47 / 5.0

Key feedback from UAT:

- Coordinators appreciated the 1-click fuzzy supervisor matching which eliminated manual name-lookup.
- Supervisors noted that the DRAFT/COMPLETED status distinction prevented accidental premature final submission.
- Students found the invitation-based group formation intuitive, with the roll number search greatly reducing formation errors.

6.2 System Performance Evaluation

API response time benchmarks were measured using Postman automated test collections with 50 sequential requests per endpoint under a single authenticated user session:

Table 6.3: API Response Time Benchmarks

Endpoint	Operation	Avg. Response (ms)	Target (< 500 ms)
POST /api/auth/login	Credential validation + JWT issuance	142 ms	Pass
GET /api/groups	Coordinator group list (paginated)	87 ms	Pass
POST /api/evaluations	Mark submission + status update	95 ms	Pass
POST /api/groups/:id/finalize	Finalization + rubric generation (transaction)	210 ms	Pass
GET /api/print/group/:id	Single group PDF generation	1,980 ms	Pass (< 2.5 s)

6.3 Functional Validation of Evaluation Metrics

1. **Functional Completeness:** All 11 functional requirements (FR1–FR11) were implemented and verified. UAT confirmed 100% task completion rate.
2. **System Performance:** All transactional API endpoints responded within 300 ms; PDF generation averaged under 2 seconds, within the 2.5-second target.
3. **Usability:** UAT aggregate satisfaction score of 4.47/5.0 exceeded the minimum target of 4.0/5.0.
4. **Mark Calculation Accuracy:** Automated mark aggregation was cross-checked against manual calculations for 20 test evaluation datasets; 100% consistency achieved.
5. **Email Domain Policy:** Domain enforcement was validated in 100% of tested account creation and bulk import scenarios.

6.4 Discussion

The implementation of TPMS successfully addressed all identified system requirements and exceeded the defined functional evaluation metrics. The multi-tier, role-isolated architecture demonstrated clear scalability benefits, as each tier can be independently scaled without architectural renegotiation. The conflict-of-interest prevention algorithm and the engagement guard collectively eliminated manual administrative auditing processes that previously consumed significant coordinator time.

The Puppeteer-based server-side PDF generation represented the most technically complex component, requiring careful management of headless browser lifecycle, font loading, page sizing, and CSS print media queries. The resulting grade sheets achieved visual and content parity with the department’s manually typeset documents, while reducing preparation time from hours to seconds.

The program-scoped audit trail provided a tamper-evident event log that coordinators found particularly valuable for monitoring late submissions and tracking supervisor allocation decisions. This audit capability directly addresses the institutional accountability requirements specified in the problem formulation.

One area of observed complexity was the coordinator response matrix deduplication logic, which required a multi-step aggregation pipeline to correctly merge multi-student group submissions. The fuzzy supervisor matching reduced average supervisor allocation time from approximately 15 minutes per group (manual process) to under 30 seconds.

7. Limitations and Future Enhancements

7.1 Current Limitations

Despite successfully achieving all stated objectives, the TPMS platform has the following current limitations that are acknowledged for institutional awareness:

- L1. Single Institutional Deployment:** The current implementation is scoped to DOECE, Pulchowk Campus. Adapting the system for multi-institution or multi-campus deployment would require a multi-tenancy architectural extension, including tenant-scoped data partitioning and centralized SSO integration.
- L2. Intranet-Only Deployment:** The initial deployment target is the departmental intranet. Exposing the system over the public internet would require additional hardening including rate limiting, DDOS mitigation, and TLS certificate management.
- L3. Static Evaluation Rubric Weightages:** The current implementation encodes the DOECE mark distribution (Minor 50, Major 100, Master 300) as application-layer constants. There is no UI for coordinators to dynamically reconfigure criterion weights for future academic sessions without code-level modification.
- L4. No Real-Time Collaboration:** The current platform does not support real-time concurrent editing (e.g., two coordinators simultaneously editing the same group record). WebSocket-based optimistic concurrency control is absent.
- L5. Limited Notification Channels:** The current notification infrastructure delivers in-app alerts and email notifications via SMTP. SMS-based notifications and push notification delivery via mobile applications are not yet implemented.
- L6. No Integrated Defense Scheduling:** The system manages evaluation rubrics and marks but does not provide an integrated defense calendar, slot booking, or

room allocation module. Defense scheduling remains an external coordination activity.

- L7. File Storage Locality:** Uploaded proposal and report documents are stored on the application server local filesystem. This limits horizontal scalability and backup flexibility compared to cloud-native object storage.

7.2 Future Enhancements

The following roadmap items are identified as high-value enhancements for subsequent development phases:

- FE1. Defense Scheduling and Calendar Integration:** Implement an integrated defense scheduling module enabling coordinators to propose defense date slots, supervisors and examiners to confirm availability, and the system to auto-generate defense timetable PDFs.
- FE2. Configurable Evaluation Rubric Builder:** Develop a UI-driven rubric configuration panel allowing coordinators to define custom criterion names, sub-criteria weights, and evaluation schemes per academic session and project type.
- FE3. Cloud Object Storage Integration:** Migrate document storage from local filesystem to cloud-native object storage (e.g., AWS S3 or equivalent), enabling geo-redundant backups, content delivery, and independent frontend/backend scaling.
- FE4. Real-Time WebSocket Notifications:** Integrate Socket.IO or native WebSocket channels to deliver instant real-time notifications on group status changes, evaluation submissions, and coordinator assignments without page refresh.
- FE5. Mobile Application (Progressive Web App):** Package the React SPA as a Progressive Web App (PWA) with offline caching, push notification support, and installable home screen presence for student and faculty convenience.
- FE6. Multi-Institution Tenancy:** Re-architect the database schema and API layer to support multi-tenant deployment, enabling other IOE campus departments (e.g., Thapathali, Purwanchal) to independently administer their project management workflows within a shared platform.

- FE7. Automated Plagiarism Similarity Check Integration:** Integrate a document similarity scoring pipeline that computes cosine similarity between submitted proposals and a corpus of prior submissions, flagging suspiciously similar documents for coordinator review prior to proposal approval.
- FE8. Supervisor Workload Balancing Dashboard:** Provide coordinators with a visual workload distribution heatmap displaying the current supervisee count per faculty member across all programs, enabling equitable supervision allocation.

8. Conclusion

The Thesis and Project Management System (TPMS) was designed, developed, and validated as a comprehensive, role-aware, production-grade web platform for digitalizing and automating the complete administrative and evaluative lifecycle of Bachelor Minor/Major projects and Master research theses at the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, Tribhuvan University.

The system successfully addresses all six primary problem areas identified in the problem formulation: decentralized record-keeping, lack of multi-project engagement controls, supervisory conflict-of-interest vulnerabilities, manual mark tabulation, cumbersome official documentation, and absence of immutable audit logging.

Key technical contributions of the project include:

- A strict **program-scoped degree isolation** architecture supporting six academic programs across Bachelor and Master tiers from a single unified deployment.
- An **automated Engagement Guard** with database-level and application-layer enforcement preventing dual student enrollment across project groups and theses.
- An **algorithmic Conflict-of-Interest Filter** that dynamically excludes project supervisors from examiner selection dropdowns, eliminating ethical evaluation conflicts.
- A **multi-tier evaluation rubric engine** implementing the DOECE grading schemes for Minor (50 marks), Major (100 marks), and Master Thesis (300 marks) evaluations.
- A **server-side headless browser PDF pipeline** using Puppeteer for instant generation of official TU Pulchowk Campus A4 evaluation sheets with number-to-words mark conversion, eliminating hours of manual typesetting.
- A **coordinator response matrix** with group deduplication and 1-click fuzzy supervisor matching, reducing per-group supervisor allocation time from 15 minutes to under 30 seconds.

- A **program-scoped immutable audit trail** providing coordinators and maintainers with complete operational traceability.

Functional validation through unit testing (36 test cases, 100% pass rate), integration testing (4 end-to-end workflows), security testing (5 attack vector scenarios), and User Acceptance Testing (4.47/5.0 average satisfaction) confirmed that the system meets and exceeds all defined functional, performance, and usability requirements.

The successful implementation of TPMS demonstrates that a full-stack Node.js/React/PostgreSQL architecture with carefully engineered domain-specific business logic can effectively replace manual, paper-based academic administration workflows in institutional engineering education. The platform is ready for intranet deployment at DOECE, Pulchowk Campus, and provides a replicable, extensible model for academic project management digitalization that can be adapted by other departments and engineering institutions with analogous administrative structures.

References

- [1] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, 8th edition, 2015.
- [2] Amy J. Ko, Thomas D. LaToza, and Margaret M. Burnett. A systematic review of research on academic project management in engineering education. *IEEE Transactions on Education*, 64(3):218–229, 2021.
- [3] David Ferraiolo, D. Richard Kuhn, and Ramaswamy Chandramouli. *Role-Based Access Control*. Artech House, Norwood, MA, 2nd edition, 2007.
- [4] Ian Sommerville. *Software Engineering*. Pearson, Harlow, England, 10th edition, 2016.
- [5] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, Irvine, CA, USA, 2000.
- [6] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, Upper Saddle River, NJ, 2017.
- [7] The PostgreSQL Global Development Group. PostgreSQL: The World's Most Advanced Open Source Relational Database. <https://www.postgresql.org/docs>, 2024. Accessed: May 2026.
- [8] Prisma Data, Inc. Prisma: Next-Generation ORM for Node.js and TypeScript. <https://www.prisma.io>, 2024. Accessed: May 2026.
- [9] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force (IETF), May 2015.
- [10] Kenneth S. Rubin. *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Addison-Wesley, Upper Saddle River, NJ, 2012.
- [11] Google Chrome Developers. Puppeteer: Headless Chrome Node.js API. <https://pptr.dev>, 2024. Accessed: May 2026.

- [12] Matthew D. Ritzer. *Moodle E-Learning Course Development*. Packt Publishing, Birmingham, UK, 4th edition, 2022.
- [13] ProQuest LLC. ProQuest ETD Administrator. <https://www.proquest.com>, 2024. Accessed: May 2026.
- [14] Turnitin LLC. Turnitin Feedback Studio - Academic Integrity Solution. <https://www.turnitin.com>, 2024. Accessed: May 2026.
- [15] Gurpreet Kaur and Amandeep Singh. Design and implementation of a web-based project management system for academic institutions. *International Journal of Computer Applications*, 105(12):37–41, 2014.
- [16] Rohit Sharma, Pankaj Kumar, and Sandeep Jain. A web-based thesis management portal for university departments. In *Proceedings of the International Conference on Computer Communication and Informatics (ICCCI)*, pages 1–6, Coimbatore, India, 2019.

Appendices

Appendix A: Prisma Database Schema (Key Models)

The complete Prisma ORM schema is maintained in `backend/prisma/schema.prisma`.

Key enum types used across the platform include:

- `Role`: MAINTAINER | COORDINATOR | SUPERVISOR | STUDENT | EXTERNAL_EXAMINER
- `BachelorProjectType`: MINOR | MAJOR
- `MasterProjectType`: PROJECT | THESIS
- `GroupStatus` / `ThesisStatus`: PENDING | ACTIVE | COMPLETED | OVERDUE | REJECTED
- `EvaluationStage`: PROPOSAL | MID_TERM | FINAL
- `EvaluationType`: SUPERVISOR | PROPOSAL_DEFENSE | MIDTERM_DEFENSE | FINAL_DEFENSE
| EXTERNAL_EXAMINER | EXTERNAL_MIDTERM | EXTERNAL_FINAL

Appendix B: Backend API Route Summary

The Express.js application mounts 21 dedicated route modules under `/api/`:

<code>/api/auth</code>	Authentication and password management
<code>/api/announcements</code>	Academic announcement management
<code>/api/student-groups</code>	Student team formation and invitations
<code>/api/groups</code>	Coordinator group lifecycle management
<code>/api/theses</code>	Master thesis lifecycle management
<code>/api/evaluations</code>	Rubric evaluation mark submission
<code>/api/proposals</code>	Document upload and retrieval
<code>/api/print</code>	PDF grade sheet generation
<code>/api/users</code>	User management and bulk import
<code>/api/supervisors</code>	Supervisor assignment operations
<code>/api/notifications</code>	In-app notification management
<code>/api/files-audit</code>	Program-scoped audit log access

Appendix C: Technology Versions

React	18.x
Vite	5.x
Node.js	18.x
Express.js	4.x
PostgreSQL	16.x
Prisma ORM	5.x
Puppeteer	21.x
SheetJS (xlsx)	0.18.x
bcryptjs	2.4.x
jsonwebtoken	9.x